

# API Contract

Every endpoint, request, and response shape the app relies on.

Read this if: backend (M1-M3) and both frontend members (M4, M5)

The interface between the frontend, FastAPI, and backend services. Don't rename endpoints or reshape responses without checking with the Team Lead first — other people are already building against these.

## Authentication

Handled by Supabase Auth. The frontend attaches the user's token to every request.

## Repository

### POST /repositories

```
Request { "github_url": "https://github.com/example/repo" }
Response { "repository_id": "uuid", "github_url": "...", "status": "created" }
```

### GET /repositories

Repositories belonging to the authenticated user.

### GET /repositories/{repository\_id}

Repository metadata and current analysis status.

## Analysis

### POST /repositories/{repository\_id}/analyze

```
Response { "analysis_job_id": "uuid", "status": "queued" }
```

### GET /analysis/{analysis\_job\_id}

```
Response { "analysis_job_id": "uuid", "status": "queued" }
```

Status values: `queued`, `running`, `completed`, `failed`.

## Files

### GET /repositories/{repository\_id}/files

File tree.

### GET /repositories/{repository\_id}/files/{path}

Source and file metadata, content where permitted.

## Historian

### POST /repositories/{repository\_id}/questions

```
Request { "question": "Why does this authentication middleware exist?" }
Response { "answer": "string", "evidence": [], "confidence": "medium", "classification": "inferred" }
```

Classification values: `verified`, `inferred`, `human_knowledge`.

## Developer Guidance

### POST /repositories/{repository\_id}/guidance

```
Request { "task": "Add a new authentication endpoint" }
Response { "task": "...", "steps": [] }
```

Each step names the relevant files/modules and the reasoning for including them.

## Risk

### GET /repositories/{repository\_id}/risks

File/module, score, contributing signals, explanation. Computed deterministically — not by an LLM.

## Knowledge Gap

**GET /repositories/{repository\_id}/knowledge-gaps**

Detected gaps.

**POST /repositories/{repository\_id}/knowledge-gaps/{gap\_id}/answer**

```
Request { "answer": "Human-provided explanation" }
```

Creates a `knowledge_items` row and links it back to the gap.

## Knowledge

**GET /repositories/{repository\_id}/knowledge**

Reusable human knowledge with its associated evidence.

## Offboarding

**POST /repositories/{repository\_id}/offboarding**

```
Request { "contributor": "contributor_identifier" }
Response { "session_id": "uuid", "status": "created" }
```

**GET /repositories/{repository\_id}/offboarding/{session\_id}**

Session questions, risk areas, and knowledge-transfer results.

## Error format

```
{ "error": { "code": "REPOSITORY_NOT_FOUND", "message": "Repository was not found." } }
```

## Integration rule

Frontend uses the backend's actual response schemas — never guesses field names. Backend modules talk through service interfaces, not each other's internals.